2. TRD — Technical Requirements Document

Project: Advanced Personal Portfolio Website + Admin Panel **Builder:** Google AI Studio (Build / Max mode) → deploy to Vercel (or any static host) with Supabase backend
Version: 1.0

> **Roman Urdu summary:** Yeh document batata hai website *kaise* banay gi — kaun si technologies, folder structure, intro video ka state machine, theme, form pipeline, security, deployment. AI Studio ko yeh document dene se woh random stack choose nahi karega.

---

1. Technology Stack

Layer	Choice	Why
Language	**TypeScript** (strict mode)	Safer refactors, AI builders handle it well
UI framework	**React 18** + **Vite**	Default for AI Studio Build; fast dev/build
Routing	**React Router v6** (data router)	Real URLs for projects, blog, admin
Styling	**Tailwind CSS** + CSS variables for design tokens	Fast, consistent, theme-able
Animation	**Motion** (Framer Motion) + CSS transitions	Declarative, reduced-motion aware
Data fetching	**TanStack Query**	Caching, loading/error states
Forms	**React Hook Form + Zod**	One schema for client validation, reused in admin
Rich text (admin)	**Tiptap**	Case-study editor, outputs sanitized HTML/JSON
Drag & drop (admin)	**dnd-kit**	Reorder projects, gallery, sections
Charts (admin)	**Recharts**	Analytics dashboard

Layer	Choice	Why
SEO tags	**react-helmet-async**	Per-route meta
Backend	**Supabase** (Postgres, Auth, Storage, Edge Functions)	One service for DB + auth + files + server code; Row Level Security
Email	**Resend** (or any SMTP/API provider)	Inquiry notifications, called only from server code
Bot protection	**Cloudflare Turnstile**	Free, privacy-friendly CAPTCHA alternative
Icons	**lucide-react**	Consistent, tree-shakeable
Testing	Vitest + Testing Library, Playwright (E2E), axe-core (a11y)	See §17

> **Constraint note:** The spec says to use whatever the AI builder supports and avoid unnecessary migration. If AI Studio provides Firebase instead of Supabase, keep the same data model (see Backend Schema doc) and map tables to collections; do not change the UI or flows.

2. High-Level Architecture

```
Browser (React SPA, pre-rendered public routes)
  ├─ Public site  ── read-only queries ──▶ Supabase REST (anon key, RLS: published rows only)
  ├─ Contact form ── POST ──▶ Edge Function `submit-inquiry` ──▶ DB (+ Resend email)
  ├─ Analytics   ── POST ──▶ Edge Function `track-event` ──▶ DB (anonymous)
  └─ Admin /admin ── JWT (Supabase Auth) ──▶ Supabase REST/Storage (RLS: is_admin())

Supabase
  ├─ Postgres (tables + RLS + triggers)
  ├─ Auth (email/password, optional TOTP)
  ├─ Storage (buckets: media, intro, documents)
  └─ Edge Functions (server secrets live here only)
```

Rules

1. The browser only ever holds the **anon key** and public config. The **service-role key, email API key, Turnstile secret** exist only in Edge Function secrets.
2. Authorization is enforced by **RLS**, never by hiding routes.
3. The app must run in **two data modes** (see §3).

3. Data Layer and Demo Mode (important for AI Studio)

AI Studio's preview often has no database connected. The app must therefore work immediately with dummy data and switch to the real backend by configuration only.

```
// src/data/provider.ts
export interface DataProvider {
  getSiteSettings(): Promise<SiteSettings>;
  listProjects(q: ProjectQuery): Promise<Paged<Project>>;
  getProjectBySlug(slug: string): Promise<ProjectDetail | null>;
  // ... one method per module (skills, services, testimonials, blog, etc.)
  submitInquiry(input: InquiryInput): Promise<{ ok: true } | { ok: false;
error: string }>;
  // admin methods: create/update/delete/reorder/publish per module
}
```

- **`SeedProvider`** — reads bundled JSON from `src/seed/*.json` (sample data) and stores admin edits in **IndexedDB** (demo-only; clearly labelled "Demo mode — changes are saved only in this browser"). `submitInquiry` in demo mode shows an honest message: *"Demo mode: message was NOT sent."* and stores it in the demo inbox. Never a fake "sent".
- **`SupabaseProvider`** — real implementation using `@supabase/supabase-js`.
- Selected at startup: if `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` exist → Supabase, else Seed (demo). A visible badge appears in demo mode and in admin.
- Demo-mode admin login uses a clearly documented demo credential defined in `.env` (`VITE_DEMO_ADMIN_EMAIL/PASSWORD`) and is **compiled out of production builds** (`import.meta.env.PROD && !VITE_ALLOW_DEMO` → disabled).

4. Project Structure

```
src/
  app/            router.tsx, providers.tsx, error-boundary.tsx
  config/         env.ts (validated with Zod), flags.ts, motion.ts, tokens.ts
  data/           provider.ts, seed-provider.ts, supabase-provider.ts,
queries.ts
  seed/           profile.json, projects.json, skills.json, services.json,
... (sample data)
  features/
    intro/        IntroOverlay.tsx, introMachine.ts, useIntroPreference.ts
    nav/          Navbar.tsx, MobileMenu.tsx, ScrollProgress.tsx,
useScrollSpy.ts
    hero/ about/ skills/ projects/ services/ testimonials/ resume/ contact/
blog/
    palette/      CommandPalette.tsx
    theme/        ThemeProvider.tsx, themeScript.ts
    i18n/         (only if language switcher enabled)
    analytics/    track.ts
    seo/          Seo.tsx, jsonld.ts
  admin/
    layout/       AdminShell.tsx, Sidebar.tsx, RequireAuth.tsx
    pages/        Dashboard, Projects, Categories, Skills, Experience,
Services,
                  Testimonials, Blog, Intro, Media, Settings, Navigation,
Seo,
                  Inbox, Analytics, SampleData, Users, AuditLog, Backup
    components/   DataTable, RichTextEditor, MediaPicker, SortableList,
ConfirmDialog
  components/ui/  Button, Card, Badge, Input, Textarea, Select, Dialog,
Drawer, Toast, Skeleton
  lib/            utils.ts, sanitize.ts, slug.ts, storage.ts (safe
localStorage), format.ts
  styles/         globals.css (tokens), prose.css, print.css
supabase/
  migrations/     0001_init.sql, 0002_rls.sql, 0003_seed_helpers.sql
  functions/      submit-inquiry/, track-event/
public/           robots.txt, favicon, og-default.jpg, seed media
placeholders
```

5. Environment Variables

Variable	Where	Purpose
`VITE_SUPABASE_URL`	client	Supabase project URL
`VITE_SUPABASE_ANON_KEY`	client	Public anon key (safe by design with RLS)
`VITE_SITE_URL`	client/build	Canonical base URL for SEO/sitemap
`VITE_TURNSTILE_SITE_KEY`	client	Public CAPTCHA key
`VITE_DEMO_ADMIN_EMAIL`, `VITE_DEMO_ADMIN_PASSWORD`, `VITE_ALLOW_DEMO`	client (dev only)	Demo admin, never in production
`SUPABASE_SERVICE_ROLE_KEY`	**server secret**	Used only inside Edge Functions
`RESEND_API_KEY`	**server secret**	Send notification emails
`NOTIFY_EMAIL_TO`, `NOTIFY_EMAIL_FROM`	server secret	Owner address / verified sender
`TURNSTILE_SECRET_KEY`	**server secret**	Verify CAPTCHA tokens
`IP_HASH_SALT`	**server secret**	Hash IPs for rate limiting without storing raw IPs

A `.env.example` file lists every variable with comments. Missing required variables must produce a clear in-app “Configuration needed” message listing exactly what to set; no silent failure.

6. Routing, Rendering and SEO

Route	Component	Notes
`/`	Home	Sections: hero, about, skills, featured projects, services, experience, testimonials, FAQ, contact. Anchors `/#about` etc.

Route	Component	Notes
`/projects`	ProjectsPage	Filters, search, sort, pagination
`/projects/:slug`	ProjectDetail	404 component if slug not found/unpublished
`/resume`	ResumePage	Print stylesheet + CV download
`/blog`, `/blog/:slug`	Blog	Behind `blog_enabled` flag
`/privacy`	Legal page	Content from admin
`/admin/*`	Admin	Lazy-loaded chunk, `RequireAuth`, `noindex`
`*`	NotFound	Custom 404

SEO approach for an SPA: (a) per-route `<Seo/>` using react-helmet-async; (b) **pre-render public routes at build time** (e.g. `vite-plugin-ssr` / `vite-ssg`-style static generation, or a prerender script that fetches published slugs) so crawlers and social scrapers receive real HTML with meta tags; (c) generate `sitemap.xml` at build from published slugs; (d) static `robots.txt` disallowing `/admin`; (e) `<meta name="robots" content="noindex">` on admin and on staging via `VITE_SITE_URL` check. If pre-rendering is not possible in the builder, document the limitation and recommend migrating the same code to a framework with SSR (Next.js) later.

Scroll restoration, focus reset to `<h1>` on route change, and `aria-live` route announcements are required.

7. Intro Video State Machine

States: `checking-preference` → `intro-loading` → `intro-playing` → `transitioning` → `portfolio-ready`, plus `intro-fallback`.

```
type IntroState =
  | 'checking-preference' | 'intro-loading' | 'intro-playing'
  | 'transitioning' | 'portfolio-ready' | 'intro-fallback';

type IntroEvent =
  | { type: 'PREF_LOADED'; skip: boolean; reducedMotion: boolean; enabled:
boolean }
  | { type: 'CAN_PLAY' } | { type: 'PLAYING' }
  | { type: 'ENDED' } | { type: 'SKIP' }
  | { type: 'ERROR' } | { type: 'TIMEOUT' } | { type: 'AUTOPLAY_BLOCKED' }
  | { type: 'TRANSITION_DONE' } | { type: 'REPLAY' };

// Pure reducer. Guards make duplicate events harmless:
// ENDED and SKIP in the same tick -> only the first one moves 'intro-
playing' -> 'transitioning'.
```

Behaviour rules

- `PREF_LOADED` → if intro disabled in admin, or visitor chose skip, or reduced motion is on → go straight to `portfolio-ready` (reduced motion: show short fade only).
- `intro-loading` has a **timeout** (default 6 s from admin settings) → `intro-fallback` (shows poster + Play button + “Enter site”) — never a dead screen.
- `ERROR`, `AUTOPLAY_BLOCKED` → `intro-fallback`.
- Transition runs from a single `useEffect` guarded by a ref (`hasTransitioned`); all listeners are removed in cleanup; replay resets refs.
- While the overlay is active: the main app is rendered underneath with `inert` + `aria-hidden`; when finished, the overlay is **unmounted** (hidden controls must not stay focusable). Focus then moves to the hero `<h1>`.
- `localStorage` keys (wrapped in `safeStorage` with try/catch): `pf:introSkip`, `pf:theme`, `pf:lang`. Nothing else is stored.
- Video element: `playsInline muted preload="metadata"` + `poster`, `<source>` for MP4 (H.264) with optional WebM, `<track kind="captions" srclang="en" default>`. Use `object-fit: cover` with `object-position` set from admin (focal point) so faces are not cropped; separate mobile source via `<source media>` or JS switch.
- Transition (Motion): overlay scales 1 → 0.92 while fading and masking into hero (clip-path circle/inset), hero background colours match the final frame (set `--intro-end-color` from admin), light-sweep gradient overlay, then stagger (navbar 0 ms, h1 150 ms, paragraph 300 ms, CTAs 450 ms). Total ≤ 1.4 s (instant when reduced motion).

8. Animation System

- `config/motion.ts` exports durations (`fast .18s` , `base .35s` , `slow .7s`), easings (`[0.22,1,0.36,1]`), distances, and stagger values. **No component defines its own random timing.**
- Reusable components/hooks: `<Reveal>` , `<Stagger>` , `<TextReveal>` , `useParallax` , `useReducedMotion()` (wraps `matchMedia`), `useIsTouch()` , `useLowPower()` (checks `navigator.hardwareConcurrency <= 4` , `deviceMemory` , `saveData`).
- Content is rendered visible in the DOM; animation only changes transform/opacity from a non-hidden start via CSS class applied by JS, so if JS animation fails nothing remains `opacity:0` (use `<noscript>` style and an `animation-ready` class set after mount).
- Infinite animations limited to hero background; pause when tab hidden or element off-screen (`IntersectionObserver`).
- 3D scene (flag `three_d_hero_enabled` , default off): lazy-loaded chunk (`React.lazy`), disabled on touch/low-power/reduced-motion.

9. Theme System

- CSS variables on `:root[data-theme="dark|light"]` . Default = system preference.
- Inline script in `index.html` `<head>` reads `pf:theme` (try/catch) and sets `data-theme` before first paint to prevent flash.
- Tailwind configured with `darkMode: ['class','[data-theme="dark"]']` and colours referencing CSS variables.

10. Contact Pipeline

1. Client validates with the shared Zod schema (name ≤ 80, email format, message 20–5000 chars, honeypot empty, consent if required).
2. Turnstile widget produces a token (hidden when `VITE_TURNSTILE_SITE_KEY` missing → admin sees warning; form then relies on honeypot + rate limit only).
3. POST to Edge Function `submit-inquiry` with `{ ...fields, turnstileToken }` .
4. Function: re-validates with the same schema, verifies Turnstile with secret, checks rate limit (e.g. 3 per hour per hashed IP and 10 per day), strips HTML, inserts into `inquiries` , sends email via Resend, returns `{ ok: true, id }` .

5. UI shows success **only** on `ok: true`. On network/validation/server error show a specific message and keep the typed data so the user can retry.
6. If the email provider fails but the DB insert succeeded, return `ok: true` and set `inquiries.notify_status='failed'` so the owner sees it in admin (message is never lost).

11. Authentication and Authorization

- Supabase Auth (email + password); optional TOTP MFA enrolment screen in admin.
- `admin_users(user_id, role)` table; SQL function `is_admin()` and `is_owner()` used by RLS policies.
- Public (anon) policies: `SELECT` only where `status='published'` and (`publish_at is null or publish_at <= now()`). Anon `INSERT` is **not** allowed on `inquiries` / `analytics_events` directly — only through Edge Functions.
- Admin policies: full CRUD for admins; editors blocked from `admin_users`, `site_settings` danger fields, deletion of inquiries and the danger zone (checked in policy via `is_owner()`).
- Client route guard (`RequireAuth`) is UX only; the real enforcement is RLS.
- Session: auto refresh, idle timeout (default 60 min) with warning dialog; sign-out clears query cache.

12. Media and Storage

- Buckets: `media` (public read; images), `intro` (public read; video, poster, captions), `documents` (public read; CV PDF), `private-exports` (admin only).
- Upload limits enforced in client **and** bucket policy: images ⩽ 5 MB (admin shows compression advice; client-side resize to max 2400 px and WebP conversion via canvas when possible), video ⩽ 50 MB (recommend ⩽ 15 MB for the intro), PDF ⩽ 10 MB, VTT ⩽ 200 KB. MIME type allow-list.
- Every image requires alt text (blocking validation in admin) except decorative images flagged as such.
- Responsive images: `srcset` / `sizes`, explicit `width` / `height` (or aspect-ratio) to prevent CLS, `loading="lazy"` below the fold, `fetchpriority="high"` for the hero image only.

- Replace-file keeps the media row id and URL (new object, updated row, cache-busting `?v=`).

13. Analytics (first-party, privacy-friendly)

- `track(event, props)` sends to Edge Function `track-event` using `navigator.sendBeacon` / `fetch keepalive`. Events: `page_view`, `project_view`, `cta_click`, `form_start`, `form_submit`, `intro_play`, `intro_skip`, `intro_complete`.
- No cookies, no raw IP, no fingerprinting; anonymous session id kept **in memory only** (not localStorage). Honour Do-Not-Track and the `analytics_enabled` flag. Never send form content.

14. Performance Budget

- Initial JS (public home) ⩽ 180 KB gzip; admin chunk lazy-loaded separately.
- Intro poster ⩽ 150 KB (WebP); intro video ⩽ 15 MB recommended; `preload="metadata"`, start fetching video after first paint.
- Fonts: max 2 families, `font-display: swap`, self-hosted subset, with metric-compatible fallbacks to stop layout shift.
- Targets: LCP < 2.5 s, CLS < 0.1, INP < 200 ms on mid-range mobile over throttled 4G. CI runs Lighthouse on `/` and `/projects`.

15. Accessibility (Technical)

- Landmarks (`header`, `nav`, `main`, `footer`), one `<h1>` per route, logical heading order.
- Skip-to-content link; visible `:focus-visible` ring (2 px accent, 3:1 contrast).
- Dialogs/drawers/menus: focus trap, Escape to close, return focus to trigger, `aria-modal`, scroll lock.
- Forms: `<label for>`, `aria-describedby` for hints/errors, `aria-invalid`, error summary focus on submit.
- Carousels: pause/play, prev/next buttons, `aria-live="off"` when auto-rotating.
- Command palette: `role="dialog"` + combobox pattern.
- Reduced motion: global CSS `@media (prefers-reduced-motion: reduce)` plus JS hook.
- Intro video: captions track, transcript text available (collapsible) for the intro script.

16. Security Requirements

- Strict **Content-Security-Policy** header (via `vercel.json` / host config): `default-src 'self'`; `connect-src` limited to Supabase + Turnstile; `img-src` / `media-src` to Supabase storage; `frame-src` Turnstile (+ booking provider if used).
- Other headers: `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy`, `X-Frame-Options: DENY` (or CSP frame-ancestors).
- **Sanitize** all rich HTML from the editor with DOMPurify on render (and strip on save); never use raw `dangerouslySetInnerHTML` without it.
- External URLs entered in admin validated (`https:` or `mailto:` / `tel:` only); outgoing links `rel="noopener noreferrer"`.
- Inquiry data: store minimum; retention setting (default 12 months) enforced by scheduled cleanup function.
- Secrets only in server env; repo contains `.env.example` only. `.gitignore` covers `.env*`.
- Admin login throttling provided by auth provider + generic error messages; audit trail for all writes (DB triggers).

17. Testing Strategy

Level	Tooling	Must cover
Unit	Vitest	introMachine reducer (all transitions, duplicate ENDED+SKIP), slug util, safeStorage failure, Zod schemas, filters/search
Component	Testing Library	Navbar/mobile menu keyboard, form states, project filters, lightbox focus
E2E	Playwright	First visit → intro → skip; video-end path (mock short video); error fallback; contact submit success/failure; project routes; 404; theme persistence; admin login → create project → publish → visible publicly
A11y	axe-core in Playwright	No serious/critical violations on every route

Level	Tooling	Must cover
Security	Manual + SQL tests	Anonymous cannot read drafts, inquiries, analytics, admin tables; editor cannot hit owner-only actions
Visual/responsive	Playwright screenshots	360, 390, 768, 1024, 1440, 1920 widths + landscape phone
Performance	Lighthouse CI	Budget in §14

Browser matrix: Chrome, Edge, Firefox, Safari desktop; iOS Safari, Chrome Android.

18. Error Handling and Observability

- Global React error boundary with friendly fallback and “Reload”; route-level boundaries so one failing section never blanks the page.
- Every query exposes loading/empty/error UI; failed images fall back to a neutral placeholder.
- Edge Functions log structured errors (no personal data). Optional Sentry hook disabled by default.
- Network failure on admin save keeps unsaved form state and shows retry.

19. Build and Deployment

1. Source in a GitHub repository; branch `main` = production, PRs get preview deployments.
2. Host on **Vercel** (or Netlify/Cloudflare Pages); set env variables in host dashboard.
3. SPA fallback rewrite to `index.html` for non-asset routes; correct cache headers (immutable hashed assets, short HTML cache).
4. Supabase: run migrations from `supabase/migrations`, deploy Edge Functions, create first Owner via a documented one-time SQL/CLI step (no default password in code).
5. DNS and HTTPS on the custom domain; `VITE_SITE_URL` updated; sitemap submitted to search consoles.
6. Staging deployments set `noindex`.
7. Backup: scheduled database backup + admin JSON export documented.

20. Definition of Technical Completion

- All env variables documented; app boots in demo mode with zero configuration.
- No secrets in client bundle (verified by searching the build output).
- RLS enabled on all tables; policy tests pass.
- Lighthouse and axe targets met; Playwright suite green.
- Any feature that needs credentials not yet supplied is reported as *“Needs configuration: <exact variable/service>”*, never as “working”.